ANÁLISE DE GÉNEROS APLICADA AO ÁUDIO COM TRANSFORMERS

Processamento de Linguagem Natural e LLM

Abstract

Design de um classificador de géneros de músicas.

Diogo Mendes

20251612

Contents

1. Contexto 2
2. Recolha de Dados 2
2.1 Dataset 2
2.2 Estratégia de amostragem 2
3. Processamento de Dados 3
3.1 Pré-processamento para Random Forest 3
3.2. Pré-processamento para Transformers (AST) 4
4. Configuração do Ambiente 5
5. Análise de Géneros Musicais (Implementação) 5
5.1. Abordagem Machine learning tradicional: Random Forest 5
5.2. Abordagem *State-of-the-Art*: Audio Spectrogram Transformer (AST) 6
6. Comparação de resultados 8
6.1. Análise ao Random Forest (Machine Learning Tradicional) 8
6.2 Análise ao AST (Deep Learning - Transformers) 8
6.3 Análise aos Testes de inferência 9
7. Conclusões 10
7.1. Conclusões Principais: 10
7.2 Trabalho Futuro 10

1. Contexto

O presente relatório detalha o desenvolvimento de um sistema de classificação de géneros musicais baseado em Inteligência Artificial. O objetivo central deste projeto foi explorar, implementar e comparar duas abordagens distintas: um modelo de *Machine Learning* tradicional baseado em extração de características acústicas (MFCC) com classificador Random Forest, e um modelo de *Deep Learning* baseado na arquitetura Transformer (Audio Spectrogram Transformer - AST).

O trabalho desenvolvido pode ser consultado em: https://github.com/diogosantosmendes/executivemaster_pln/tree/main/tp3

O *dataset* utilizado está disponível em: https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification

2. Recolha de Dados

2.1 Dataset

Para o treino e validação dos modelos, foi utilizado o **dataset GTZAN**, um standard na comunidade de *Music Information Retrieval* (MIR).

- **Composição:** O dataset contém áudios distribuídos de forma equilibrada por 10 géneros musicais distintos (blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock).
- **Dados de Teste "Cego" (*Wild*):** Para além do dataset oficial, foi recolhido um conjunto de 12 músicas reais no formato .mp3 (abrangendo artistas desde Beethoven a Eminem e Metallica) para testar a capacidade de generalização dos modelos em dados do mundo real não vistos no treino.

2.2 Estratégia de amostragem

Foram definidos dois cenários distintos:

- **Traditional ML:** 99 *[1] ficheiros por género (total = 990)
- **AST:** 20 ficheiros por género (total = 200)

A função *load_data()* garante:

- Amostragem uniforme: Selecionando o mesmo montante de ficheiros por género.
- Balanceamento entre classes: pois é conhecido que vários ficheiros são clipes distintos da mesma música, assim, mediante o número de ficheiros por género, são selecionados os ficheiros de forma interpolada para evitar ao máximo cliques da mesma música para seleções mais reduzidas.
- Verificação de integridade dos ficheiros: ao validar que o ficheiro tem conteúdo.

---

*[1] Não foram utilizados todos os ficheiros de música (1000) porque o ficheiro *jazz.00054.wav* estava corrupto e assim decidiu-se remover 1 ficheiro por género para manter a uniformidade.

```python
import glob
import os
from configs import DATASET_FOLDER, GENRES

def load_data(files_per_genre):
    audio_paths = []
    labels = []

    genre_to_id = {genre: i for i, genre in enumerate(GENRES)}

    for genre in GENRES:
        folder_path = os.path.join(DATASET_FOLDER, genre, "*.wav")
        files = glob.glob(folder_path)

        entries = list(range(len(files)))
        jump = len(entries) // files_per_genre if files_per_genre > 0 else 1
        count = 0

        for idx in entries[::jump]:
            file = files[idx]
            if os.path.getsize(file) > 0:  # Integrity check: ensure file is not empty
                audio_paths.append(file)
                labels.append(genre_to_id[genre])
                count += 1
                if count >= files_per_genre:
                    break


    return audio_paths, labels
```

3. Processamento de Dados

Ambos os métodos de pré-processamento permitiam extrair vários vetores do mesmo clip, recebendo por parâmetro um conjunto de valores que representam os pontos relativos de inicio de cada amostra a processar. Para efeitos de treino foi sempre extraído apenas uma amostra por áudio a começar do inicio, mas esta estratégia permitiu mais tarde submeter várias amostras de momentos distintos do mesmo áudio ao modelo para depois verificar o género mais recorrente.

3.1 Pré-processamento para Random Forest

Utilizando a biblioteca *librosa* foi possível a leitura do ficheiro áudio e a extração de features MFCC (Mel-Frequency Cepstral Coefficients). Foram extraídos 20 coeficientes que representam o envelope espectral do som. De seguida foi calculada a média dos MFCCs ao longo do tempo, reduzindo a dimensão e criando um vetor estático de características para os clips de 30 segundos.

```python
def _extract_mfcc(self, file_list, audio_start_samples=[0]):
    features = []
    for f in file_list:
        y, sr = librosa.load(f, sr=None)

        n_samples = len(y)
        for audio_start in audio_start_samples:
            start_sample = int(n_samples * audio_start)
            end_sample = int(start_sample + TRADITIONAL_ML_AUDIO_DURATION_SECONDS * sr)
            end_sample = min(end_sample, len(y))

            sample = y[start_sample:end_sample]

            mfcc = librosa.feature.mfcc(y=sample, sr=sr, n_mfcc=20)
            features.append(np.mean(mfcc.T, axis=0))

    return np.array(features)
```

3.2. Pré-processamento para Transformers (AST)

A leitura dos ficheiros áudio para pré-processamento com vista em aplicar no AST foi efetuada por meio da biblioteca *torchaudio*. De seguida foram convertidos para formato *mono* e reamostrados para 16kHz utilizando *torchaudio.transforms.Resample*, uma exigência estrutural do modelo AST pré-treinado.

O modelo AST foi configurado para receber *inputs* fixos de 10 segundos. A extração de feeatures foi realizada por meio do *ASTFeatureExtractor* da biblioteca *tranformer*.

Por fim, para garantir a similaridade de tamanho dos vetores, os mais curtos sofreram *padding* (preenchimento com zeros), e os mais longos foram cortados.

```python
def _extract_features(self, audio_paths, labels, audio_start_samples=[0]):
    feature_extractor = ASTFeatureExtractor.from_pretrained(AST_MODEL_NAME)
    features=[]

    for idx, path in enumerate(audio_paths):
        waveform, sr = torchaudio.load(path)

        # Convert to mono if stereo
        if waveform.shape[0] > 1:
            waveform = torch.mean(waveform, dim=0, keepdim=True)

        # Resample to 16kHz (AST Requirement)
        if sr != 16000:
            resample = torchaudio.transforms.Resample(sr, 16000)
            waveform = resample(waveform)

        sr = 16000
        target_length = sr * AST_AUDIO_DURATION_SECONDS

        if waveform.shape[1] > target_length:
            n_samples = waveform.shape[1]
            for audio_start in audio_start_samples:
                start_sample = int(n_samples * audio_start)
                end_sample = int(start_sample + AST_AUDIO_DURATION_SECONDS * sr)
                end_sample = min(end_sample, waveform.shape[1])
                sample = waveform[:, start_sample:end_sample]

                inputs = feature_extractor(
                    sample.squeeze().numpy(),
                    sampling_rate=sr,
                    return_tensors="pt"
                )

                features.append({
                    "input_values": inputs.input_values.squeeze(0),
                    "labels": torch.tensor(labels[idx])
                })
        else:
            padding = target_length - waveform.shape[1]
            waveform = torch.nn.functional.pad(waveform, (0, padding))

            inputs = feature_extractor(
                waveform.squeeze().numpy(),
                sampling_rate=sr,
                return_tensors="pt"
            )

            features.append({
                "input_values": inputs.input_values.squeeze(0),
                "labels": torch.tensor(labels[idx])
            })

    feature_extractor.save_pretrained(self.model_path)
    return features
```

4. Configuração do Ambiente

Para viabilizar este projeto de Processamento de Sinais e *Machine Learning*, foi configurado um ambiente Python robusto com as seguintes dependências:

- **transformers (Hugging Face):** Para importar a arquitetura ASTForAudioClassification e o respetivo extrator de características.
- **torch e torchaudio:** Framework base para operações tensoriais, aceleração por GPU (CUDA), carregamento de ficheiros de áudio nativos e manipulação do modelo AST.
- **scikit-learn:** Para a divisão de dados de treino/teste de forma estratificada e implementação do algoritmo *Random Forest*.
- **librosa:** Para a análise acústica avançada no modelo baseline.

Foi desenvolvido o ficheiro configs.py com as configurações que podem ser alteradas entre execuções para comparação e análise das duas abordagens.

```python
# --------------------------------
# Global configs
# --------------------------------
# dataset folder path
DATASET_FOLDER = "genres_original"
# list of genres (must match the folder names in the dataset)
GENRES = ("blues", "classical", "country", "disco", "hiphop", "jazz", "metal", "pop", "reggae", "rock")
# testing split ratios to evaluate model performance
TESTING_SPLIT_RATIOS = [0.05, 0.5, 0.7, 0.8, 0.9]

# --------------------------------
# Traditional ML configs
# --------------------------------

# filename for the saved model

# number of files to load per genre for Traditional ML (to avoid imbalance)
TRADITIONAL_ML_FILES_PER_GENRE = 99
# audio duration in seconds for Traditional ML input
TRADITIONAL_ML_AUDIO_DURATION_SECONDS = 30

# --------------------------------
# AST configs
# --------------------------------

# pretrained AST model name (from Hugging Face)
AST_MODEL_NAME = "MIT/ast-finetuned-audioset-10-10-0.4593"
# number of files to load per genre for AST training
AST_FILES_PER_GENRE = 20
# audio duration in seconds for AST input (AST requires fixed-length inputs, is recommended 10 seconds)
AST_AUDIO_DURATION_SECONDS = 10
# number of epochs to train the AST model
AST_EPOCHS = 10
```

5. Análise de Géneros Musicais (Implementação)

5.1. Abordagem Machine learning tradicional: Random Forest

O modelo tradicional baseou-se num classificador *Random Forest* com 100 árvores de decisão (n_estimators=100).

O modelo mapeou as matrizes de MFCCs para as *labels* correspondentes. A simplicidade deste modelo permitiu um treino quase instantâneo, focado puramente nas características de timbre da música.

```python
def train(self, files_per_genre=TRADITIONAL_ML_FILES_PER_GENRE):
    audio_paths, labels = load_data(files_per_genre)
    print(f"Total audio files loaded: {len(audio_paths)}")

    print("--- Data Split ---")
    train_paths, test_paths, y_train, y_test = train_test_split(
        audio_paths, labels, test_size=0.2, random_state=42, stratify=labels
    )

    print("--- Extracting Features for Traditional ML ---")
    X_train = self._extract_mfcc(train_paths)
    X_test = self._extract_mfcc(test_paths)

    print("--- Training with Traditional ML (Random Forest) ---")
    self.model.fit(X_train, y_train)
    joblib.dump(self.model, self.model_name)
```

5.2. Abordagem *State-of-the-Art*: Audio Spectrogram Transformer (AST)

Foi utilizado o modelo pré-treinado *MIT/ast-finetuned-audioset-10-10-0.4593*. Este modelo baseia-se num *Vision Transformer* (ViT) adaptado para áudio, onde o som é convertido num espectrograma que é dividido em pequenos *patches* (10x10) e processado com mecanismos de *Self-Attention*.

O código foi desenvolvido para suporte automático de GPU (CUDA) se disponível, caso contrário, será executado em CPU.

De forma a obter o melhor resultado possível para os recursos limitados de computação local considerou-se o seguinte:

- Otimização Avançada: Implementou-se o otimizador AdamW com *weight decay* e um agendador de taxa de aprendizagem *Cosine Annealing* com *warmup*, garantindo estabilidade no *Fine-Tuning*.
- Prevenção de *Overfitting*: Devido ao menor volume de dados usado nesta fase (20 ficheiros por género), aplicou-se *Label Smoothing* na função de perda (CrossEntropyLoss) e mecanismos de *Early Stopping* (paciência = 4 ciclos).
- Treino Eficiente: Utilização de precisão mista (AMP - torch.amp.autocast) para acelerar o treino na GPU.

```python
def train(self, files_per_genre=AST_FILES_PER_GENRE):
    audio_paths, labels = load_data(files_per_genre)
    print(f"Total audio files loaded: {len(audio_paths)}")

    print("--- Extracting Features for Transformer (AST) ---")
    X = self._extract_features(audio_paths, labels)

    print("--- Data Split ---")
    X_train, X_tmp, _, y_tmp = train_test_split(
        X, labels, test_size=0.2, random_state=42, stratify=labels
    )
    X_val, X_test, _, _ = train_test_split(
        X_tmp, y_tmp, test_size=0.5, random_state=42, stratify=y_tmp
    )

    print("--- Loading Transformer model (AST) ---")
    genre_to_id = {genre: i for i, genre in enumerate(GENRES)}
    id_to_genre = {i: genre for genre, i in genre_to_id.items()}

    self.model = ASTForAudioClassification.from_pretrained(
        AST_MODEL_NAME,
        num_labels=len(GENRES),
        ignore_mismatched_sizes=True,
        id2label=id_to_genre,
        label2id=genre_to_id
    )
    self.model.to(self.device)

    print("--- Training Transformer (AST) ---")

    train_loader = DataLoader(X_train, batch_size=2, shuffle=True, pin_memory=True)
    val_loader = DataLoader(X_val, batch_size=2, pin_memory=True)

    # Use AdamW optimizer with weight decay and a cosine learning rate scheduler with warmup
    # to improve training stability and convergence for the AST model.
    optimizer = torch.optim.AdamW(self.model.parameters(), lr=5e-5, weight_decay=1e-4)
    # Use label smoothing in the loss function to help prevent overfitting and improve generalization of the AST model.
    criterion = torch.nn.CrossEntropyLoss(label_smoothing=0.1)

    num_training_steps = AST_EPOCHS * len(train_loader)
    num_warmup_steps = int(0.2 * num_training_steps)

    scheduler = get_cosine_schedule_with_warmup(
        optimizer,
        num_warmup_steps=num_warmup_steps,
        num_training_steps=num_training_steps
    )

    patience = 4
    counter = 0
    best_val_acc = 0

    scaler = torch.amp.GradScaler()

    for epoch in range(AST_EPOCHS):
        self.model.train()

        for batch in train_loader:
            inputs = batch["input_values"].to(self.device, non_blocking=True)
            labels = batch["labels"].to(self.device, non_blocking=True)

            optimizer.zero_grad(set_to_none=True)

            with torch.amp.autocast(device_type=self.device.type):
                outputs = self.model(input_values=inputs)
                loss = criterion(outputs.logits, labels)

            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)

            scaler.step(optimizer)
            scaler.update()
            scheduler.step()

        val_acc = self.evaluate(val_loader)["acc"]
        print(f"Epoch {epoch} done | Val Acc: {val_acc:.4f}")

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            self.model.save_pretrained(self.model_path)
            counter = 0
        else:
            counter += 1

        if counter >= patience:
            print("Early stopping!")
            break
```

Durante a fase de treino com o modelo AST verificou-se que o *early stoping* foi ativado preservando o modelo no estado do 3º ciclo. Prevenindo assim o *overfitting*, pois nos ciclos seguintes não apresentou melhorias de *accuracy*.

CICLO	ACCURACY
0	0.6000
2	0.6500
3	0.7500
5-7	0.7500

6. Comparação de resultados

6.1. Análise ao Random Forest (Machine Learning Tradicional)

O modelo começa com uma performance bastante razoável (68% de Accuracy/F1 com 50 músicas) e vai degradando de forma linear até aos 57% com 99 músicas.

Random Forest			
Metric	songs/genre		
	50	70	99
Accuracy	0,68	0,66	0,57
Precision	0,69	0,65	0,56
Recall	0,68	0,66	0,57
F1 Score	0,68	0,64	0,56

Uma possível justificação seria um problema da variância (Capacity Bottleneck). Quando se usa apenas 50 músicas por género, a diversidade dentro de cada género é menor, e o modelo consegue decorar ou encontrar "atalhos" nos dados para separar as classes. Quando se passa para 99 músicas, a verdadeira complexidade e variabilidade de cada género musical vem ao de cima. Uma simples média de MFCCs não tem poder expressivo para capturar nuances tão complexas. O modelo tenta englobar demasiados perfis sonoros diferentes debaixo da mesma *label* e acaba por se confundir, gerando mais falsos positivos e negativos.

6.2 Análise ao AST (Deep Learning - Transformers)

Um pico de performance aos 15 áudios por género (93% de F1-Score), seguido de um "colapso" inexplicável aos 20 áudios (cai para 59%).

AST			
Metric	songs/genre		
	10	15	20
Accuracy	0,90	0,93	0,60
Precision	0,85	0,97	0,57
Recall	0,90	0,93	0,60
F1 Score	0,87	0,93	0,59

Nestas quantidades, o modelo está a beneficiar imenso do facto de ser pré-treinado. Ele apenas precisa de alguns exemplos do teu *dataset* para ajustar a camada final (classificador) sem "esquecer" o que já sabia sobre áudio.

A queda abrupta aos 20 ficheiros pode ser causada pelo aumento do tamanho do *dataset* sem reajuste dos hiperparâmetros (como o *Learning Rate*, o *Warmup Steps* ou o tamanho do *Batch*), tornando o treino instável. Principalmente o tamanho do *Batch* que foi sempre 2 (por motivos de capacidade computacional não foi possível aumentar), o gradiente de atualização com 200 ficheiros no total pode ter ficado demasiado ruidoso, levando o otimizador a divergir.

6.3 Análise aos Testes de inferência

Foram realizados testes de inferência com músicas conhecidas para comparar as previsões de cada um dos modelos.

Music	Random Forest	AST
B B King - The Thrill Is Gone	"reggae"	"jazz"
Beethoven - Fur Elise	"classical"	"classical"
Bob Marley & The Wailers - Three Little Birds	"reggae"	"reggae"
Britney Spears - Baby One More Time	"jazz"	"pop"
Daft Punk - Harder, Better, Faster, Stronger	"classical"	"hiphop"
Eminem - Lose Yourself	"reggae"	"hiphop"
Gloria Gaynor - I Will Survive	"reggae"	"disco"
Guns N' Roses - Civil War	"jazz"	"country"
John Denver - Take Me Home, Country Roads	"jazz"	"country"
João Gilberto & Stan Getz - The Girl From Ipanema	"jazz"	"jazz"
Metallica - Enter Sandman	"blues"	"metal"
Star Wars- The Imperial March	"jazz"	"classical"

Análise das inferências:

- AST apresenta maior coerência musical.
- Teoricamente falhou a “Guns N' Roses - Civil War”, que idealmente seria identificado como rock, mas esta música em particular tem mais trabalho de guitarra acústica e piano, daí não ser totalmente descabida a confusão com “country”
- “Daft Punk - Harder, Better, Faster, Stronger” é classificado como musica eletrónica que é um género que não foi ensinado aos modelos, por isso os estilos mais esperados seriam “rock”, “hiphop” ou “pop”, Desta forma a inferência do AST foi a mais acertiva.
- Random Forest apresenta confusões frequentes e uma tendência para prever jazz e reggae.

7. Conclusões

7.1. Conclusões Principais:

Existem limitações do ML Tradicional, que embora os métodos estatísticos com MFCCs sejam leves e rápidos de treinar, eles carecem de abstração. O Random Forest decorou as características de baixa qualidade do dataset original, mas foi incapaz de as traduzir para o áudio de alta fidelidade contemporâneo.

O Poder do *Transfer Learning* evidenciou o potencial do modelo Transformer (AST), que compensou a falta de dados de treino através de *Transfer Learning*. A arquitetura baseada em atenção permitiu que o modelo se focasse nas características definidoras de cada género (como o timbre das guitarras distorcidas do Metal ou as secções de cordas na música Clássica), independentemente da qualidade da gravação ou dos artefactos sonoros.

7.2 Trabalho Futuro

Os resultados obtidos abrem caminho para futuras iterações e melhorias no sistema de classificação:

1. **Sintonia de Hiperparâmetros (*Hyperparameter Tuning*):** Para estabilizar o modelo AST com o *dataset* completo (99 músicas por género), sugere-se uma otimização exaustiva da taxa de aprendizagem (*learning rate*) com decaimento dinâmico, bem como a adequação do tamanho do *Batch* para suavizar o cálculo dos gradientes.
2. **Análise de Dados Multimodais:** Uma vez que a música é uma composição de melodia e letras, uma arquitetura futura poderia aliar o modelo AST (processamento de áudio) a um modelo de Processamento de Linguagem Natural (NLP), como o BERT, para classificar os géneros cruzando a textura sonora com a análise de sentimentos e a semântica das letras extraídas.
3. **Classificação binária por género:** Uma alternativa para simplificar a identificação de géneros seria criar um modelo por género, cuja função seria identificar apenas se faz ou não parte do género, assim poderá ser mais simples para o modelo a separação de classes e a otimização na identificação das características relevantes de cada género.